



Rapport projet :Module OWASP

Test d'intrusion de l'application web « OWASP Juice Shop »

Réalisée par :

Ben Jemaa Roua

Dkhil Mohamed Mahdi

Classe:

4-ING-SSIRF-B

Année universitaire

2025-2026

Chapitre1 : Collecte d'information

I. Introduction :

Ce chapitre détaille la phase de reconnaissance passive et active effectuée sur la cible. L'objectif était d'identifier la surface d'attaque, les services exposés et les technologies sous-jacentes sans déclencher d'alertes intrusives

II. Synthèse des découvertes :

L'analyse initiale a permis d'identifier de manière certaine l'application cible et son contexte d'exécution.

Hôte : localhost

Application identifiée : OWASP Juice Shop

Type d'application : Application web

Port ouvert : 3000

Service principal : HTTP.

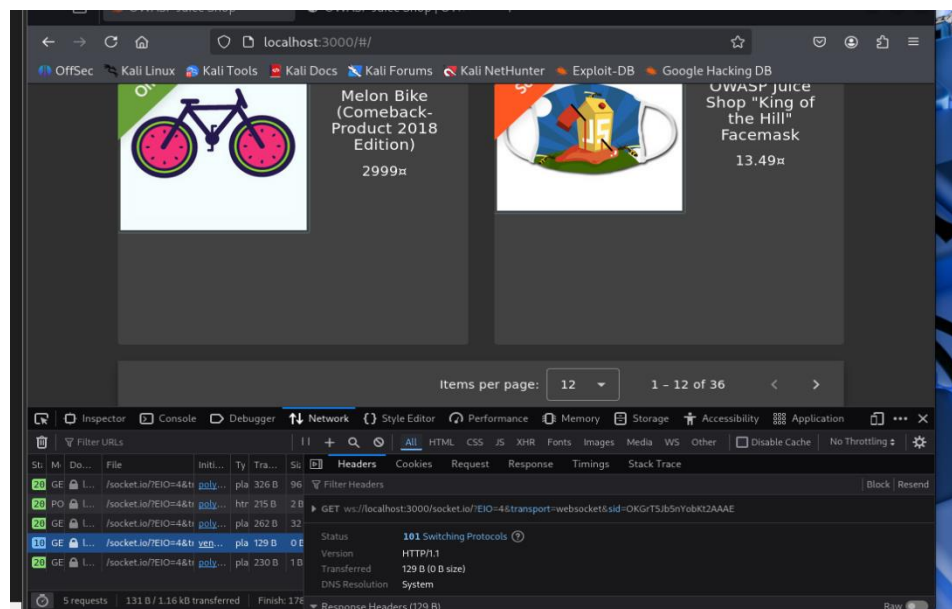


Figure 1 :Identification de l'application via l'inspecteur HTML

III. Scan de ports et détection de services :

Une analyse du réseau a été réalisée à l'aide de l'outil Nmap pour identifier les services exposés.

Commande exécutée :

```
nmap -sV -p 3000 localhost
```

Résultats :

- État du port : 3000/tcp est ouvert.
- Service : http
- Bannière / Fingerprint : Bien que le service ne soit pas explicitement nommé "Node.js" par Nmap, la réponse HTTP suivante a été récupérée, confirmant la nature du service

|« HTTP/1.1 200 OK

Content-Type: text/html »

Conclusion :

Un serveur web est actif et accessible sur le port 3000.

```
kali@kali: ~  
Session Actions Edit View Help  
  
$ nmap -sV -p 3000 localhost  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-27 06:28 EST  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00013s latency).  
Other addresses for localhost (not scanned): ::1  
  
PORT      STATE SERVICE VERSION  
3000/tcp open  ppp?  
1 service unrecognized despite returning data. If you know the service/version, please submit  
fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :  
SF:Port3000-TCP:V=7.95I=7%D=2/27Time=69A17FEEP=x86_64-pc-linux-gnu%(Ge  
SF:tRequest,10189,"HTTP/1..1\\x20200\\x200K\\r\\nAccess-Control-Allow-Origin:\\  
SF:x20*\\r\\nX-Content-Type-Options:\\x20nosniff\\r\\nX-Frame-Options:\\x20SAME  
SF:ORIGIN\\r\\nFeature-Policy:\\x20payment\\x20'self'\\r\\nX-Recruiting:\\x20/#/j  
SF:obs\\r\\nAccept-Ranges:\\x20bytes\\r\\nCachE-Control:\\x20public,\\x20max-age=  
SF:0\\r\\nLast-Modified:\\x20Fri,\\x2027\\x20Feb\\x202026\\x2011:12:00\\x20GMT\\r\\n  
SF:Ftag:\\x20W/\\x201252f-19c9ecc603c\\x20\\r\\nContent-Type:\\x20text/html;\\x20char  
SF:set=UTF-8\\r\\nContent-Length:\\x2075055\\r\\nVary:\\x20Accept-Encoding\\r\\nDa  
SF:te:\\x20Fri,\\x2027\\x20Feb\\x202026\\x2011:28:46\\x20GMT\\r\\nConnection:\\x20c  
SF:lose\\r\\n\\r\\n-----\\n\\x20\\x20-x20--\\x20Copyright\\x20(c)\\x202014-2026\\x20Bioer
```

Figure 2 : Résultat du scan Nmap

IV. Reconnaissance de l'application:

L'analyse du contenu HTML de la page d'accueil a permis d'identifier formellement l'application.

Résultats :

```
<title>OWASP Juice Shop</title>

<meta name="description" content="Probably the most modern and sophisticated
insecure web application">
```

Conclusion :

Donc confirmation que c'est bien l'application OWASP Juice Shop

V. Analyse des en-têtes HTTP et technologies front-end:

L'inspection des en-têtes de réponse et du code source a révélé des informations précieuses sur la configuration du serveur et les technologies utilisées.

V.1. Technologies Front-end :

Commande exécutée :

```
whatweb http://localhost:3000
```

Résultats :

- HTML5 : Utilisation de la dernière version du standard HTML.
- jQuery : Version 2.2.4 détectée. Cette version est ancienne et peut contenir des vulnérabilités connues.

```
(kali㉿kali)-[~]
$ whatweb http://localhost:3000
http://localhost:3000 [200 OK] HTML5, IP[::1], JQuery[2.2.4], Script[module], Title[OWASP Juice Shop],
UncommonHeaders[access-control-allow-origin,x-content-type-options,feature-policy,x-recruiting], X-Fram
e-Options[SAMEORIGIN]
```

Figure 3 : Résultat du whatweb

V.2. En-têtes de sécurité :

Commande exécutée :

```
Curl -I http://localhost:3000
```

Résultats :

- X-Frame-Options: SAMEORIGIN : Protection contre le clickjacking activée.
- Access-Control-Allow-Origin: * : Configuration CORS permissive (wildcard), ce qui pourrait être un point d'entrée pour certaines attaques.
- X-Recruiting: /#/jobs : Un en-tête personnalisé intéressant, révélant potentiellement une route ou une fonctionnalité de l'application.

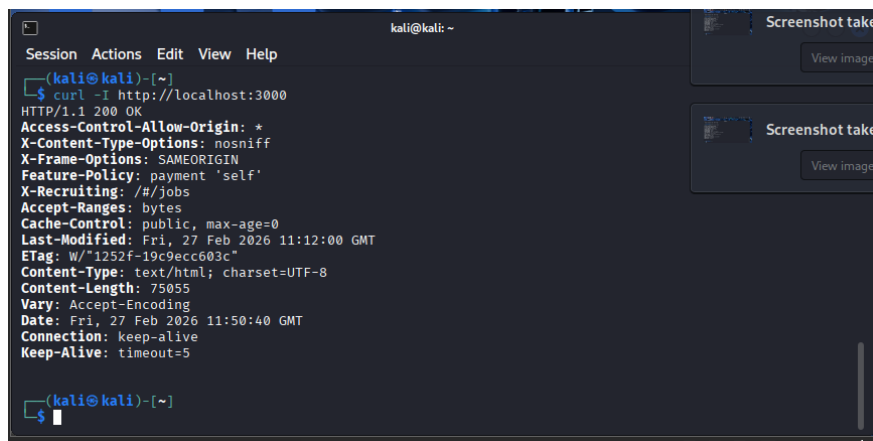


Figure 4: En-têtes HTTP

VI. Exploration des fichiers sensibles (robots.txt):

La requête du fichier robots.txt permet d'identifier les répertoires que l'administrateur souhaite dissimuler aux robots d'indexation.

Commande exécutée :

```
curl http://localhost:3000/robots.txt
```

Résultats :

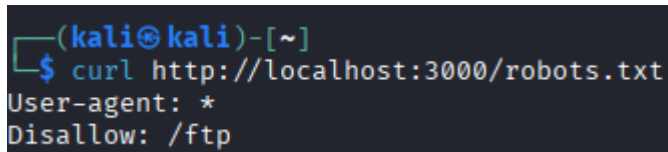
```
User-agent: *
```

Disallow: /ftp

Conclusion :

Le fichier robots.txt révèle un répertoire d'intérêt :/ftp

Ce répertoire pourrait contenir des fichiers accessibles publiquement ou mal configurés.



```
(kali㉿kali)-[~]  
$ curl http://localhost:3000/robots.txt  
User-agent: *  
Disallow: /ftp
```

Figure 5: robots.txt

VII. Détection du pare-feu applicatif (WAF):

L'outil wafw00f a été utilisé pour déterminer si un WAF (Web Application Firewall) protège l'application.

Commande exécutée :

Wafw00f http://localhost:3000

Résultats :

Aucun WAF détecté.

Conclusion :

L'application ne semble pas protégée par un pare-feu applicatif, ce qui pourrait faciliter certaines attaques comme les injections ou le scanning de répertoires.

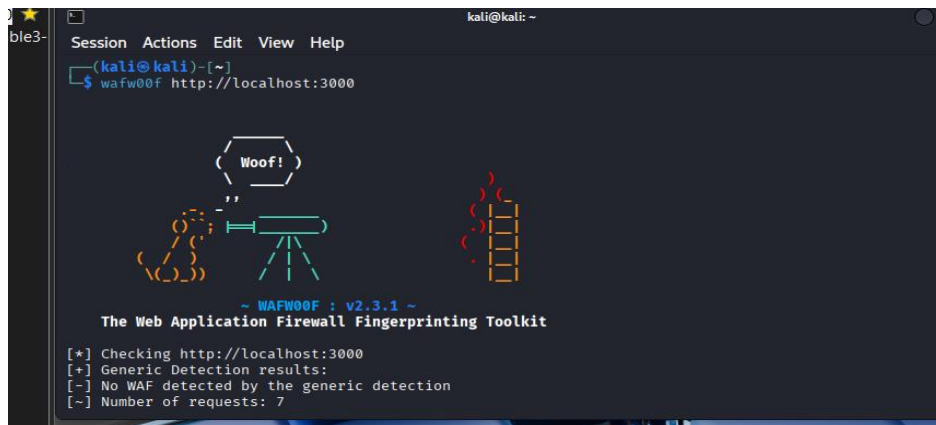


Figure 6: Outil wafw00f

VIII. Synthèse des informations collectées:

Le tableau ci-dessous récapitule l'ensemble des éléments découverts lors de cette phase.

Elément	Information Collectée	Détails
Hôte	localhost	-
Port ouvert	3000/tcp	Service HTTP
Service	HTTP	Serveur web actif
WAF	Aucun	Détecté avec wafw00f
Fichier sensible	robots.txt présent	Contient une règle Disallow
Répertoire découvert	/ftp	Via robots.txt
Technologie front-end	jQuery	Version 2.2.4
CORS	* (wildcard)	Configuration permissive
Anti-clickjacking	SAMEORIGIN	Protection activée

En-tête personnalisé	X-Recruiting: /#/jobs	Information sur une route existante
-----------------------------	-----------------------	--

Tableau 1: Synthèse des informations collectées

IX. Conclusion :

La phase d'information gathering a rempli ses objectifs en cartographiant la surface d'attaque de l'application cible. Les informations collectées permettent désormais de disposer d'une vision claire des premières failles potentielles

Chapitre 2 : Identification et exploitation des vulnérabilités

I. Introduction :

Suite à la phase de collecte d'informations qui nous a permis de cartographier la surface d'attaque de l'application OWASP Juice Shop, la phase d'**identification des vulnérabilités** représente l'étape cruciale où les informations recueillies sont exploitées pour découvrir des failles de sécurité exploitables.

II. Résumé des vulnérabilités et Bulletin d'évaluation:

Les tableaux suivants illustrent les vulnérabilités découvertes, classées par impact, ainsi que les correctifs recommandés :

Constat	Gravité	Recommandation
JUICY-001 : Injection SQL	Critique	Utiliser des requêtes paramétrées et valider les entrées
JUICY-002 : Injection SQL (recherche)	Critique	Utiliser des requêtes paramétrées pour tous les accès à la base de données
JUICY-003 : Mot de passe admin faible	Critique	Imposer une politique de mots de passe forts et activer le MFA
JUICY-004 : Exposition de données confidentielles - Répertoire FTP	Elevée	Restreindre l'accès au répertoire /ftp ; mettre en place une authentification
JUICY-005 : Broken Access Control (panier)	Elevée	Contrôler l'autorisation côté serveur avant tout accès à une ressource

Constat	Gravité	Recommandation
JUICY-006 : Accès page Admin	Elevée	Implémenter un RBAC côté serveur ; ne pas exposer les routes admin en JavaScript
JUICY-007 : Exposition du tableau des scores	Moyenne	Supprimer ou protéger par mot de passe le tableau des scores en production
JUICY-008 : Exposition de la documentation de l'API	Moyenne	Restreindre l'accès à la documentation de l'API au personnel autorisé
JUICY-009 : XSS – Champ de recherche	Moyenne	Encoder les sorties HTML et valider les entrées ; déployer une Content Security Policy (CSP)
JUICY-010 : CSRF Feedback	Moyenne	Déduire le UserId depuis le token de session ; implémenter des jetons CSRF

III. Résultats du Test d'Intrusion Interne:

1. Constat JUICY-001 : Injection SQL (Critique)

Description	Juice Shop (OWASP) permet l'injection SQL dans le formulaire de connexion. TCMS a réussi à contourner l'authentification en utilisant la charge utile ' OR 1=1-- dans le champ email, avec n'importe quel mot de passe.
Risque	Probabilité : Élevée – L'injection SQL est un vecteur d'attaque bien connu, avec des outils automatisés disponibles. Impact : Très élevé – Une exploitation réussie permet un contournement complet de l'authentification et une extraction potentielle des données de la base de données.
Composant affecté	Formulaire de connexion (/login)
Outils utilisés	Tests manuels
Références	OWASP SQL Injection, NIST SP800-53 r4 SI-10 - Information Input Validation (Validation des entrées d'informations)

Preuve :

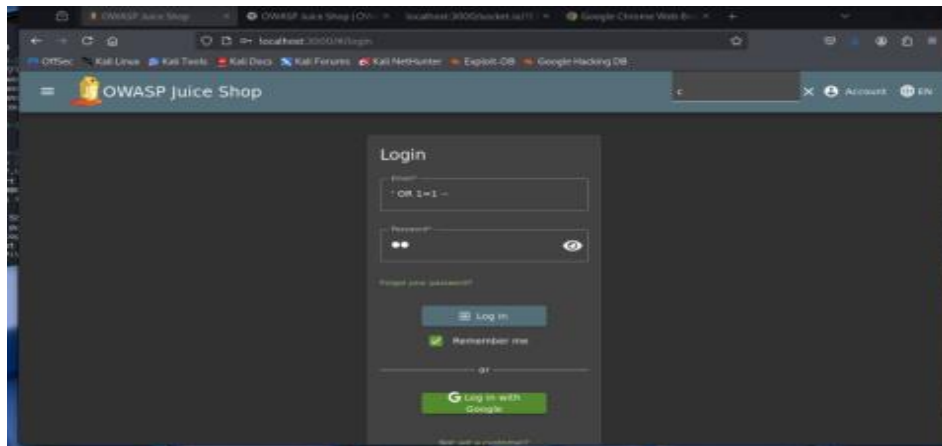


Figure 1 : Charge utile d'injection SQL 'OR 1=1--' dans le champ email

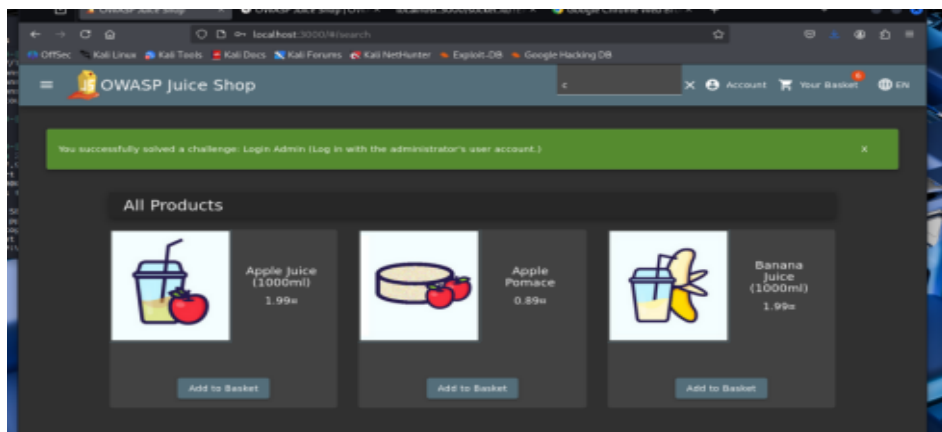


Figure 2 : Contournement de l'authentification réussi

Mesures correctives:

- Mettre en œuvre des requêtes paramétrées ou des instructions préparées pour toutes les interactions avec la base de données.
- Utiliser une bibliothèque de mappage objet-relationnel (ORM) qui gère automatiquement la paramétrisation.
- Appliquer une validation stricte des entrées et envisager l'utilisation d'un pare-feu pour applications Web (WAF) comme mesure d'atténuation temporaire.

2. Constat JUICY-002 : Injection SQL – Champ de recherche (Critique)

Description	Le paramètre de recherche q dans /rest/products/search est vulnérable à une injection SQL. En injectant la charge utile ')) UNION SELECT ... dans le champ de recherche, il est possible d'extraire le contenu de la base de données SQLite, notamment les tables utilisateurs et leurs mots de passe hachés.
Risque	Probabilité : Élevée – La vulnérabilité est exploitable manuellement ou via des outils automatisés comme sqlmap. Impact : Critique – Un attaquant peut extraire l'intégralité de la base de données, incluant les identifiants et mots de passe de tous les utilisateurs.
Composant affecté	Barre de recherche : <code>http://localhost:3000/rest/products/search?q=))UNION SELECT id,email,password,'4','5','6','7','8','9' FROM Users--</code>
Outils utilisés	Tests manuels
Références	OWASP Top 10 A3:2021-Injection,

Preuve :

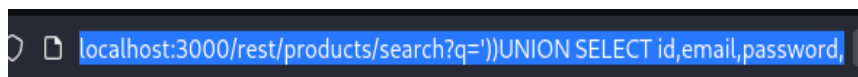


Figure 3 : Injection SQL UNION sur /rest/products/search

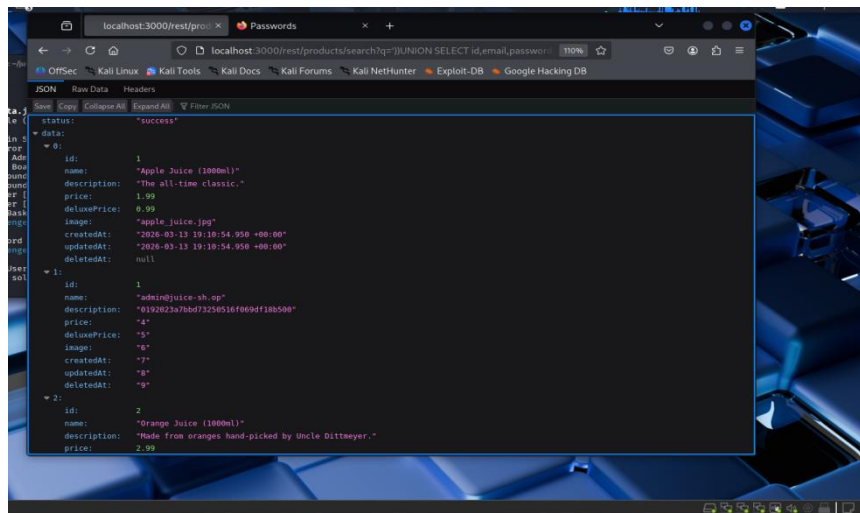


Figure 4 : extraction des emails et mots de passe hashés de la table Users

Mesures correctives :

- Utiliser exclusivement des requêtes paramétrées (prepared statements) pour toutes les interactions avec la base de données.
- Mettre en place une validation stricte et un assainissement des entrées utilisateur avant tout traitement.
- Appliquer le principe du moindre privilège sur le compte de base de données utilisé par l'application.

3. Constat JUICY-003 : Authentification insuffisante – Mot de passe administrateur faible (Critique)

Description	Après craquage de hash , le compte administrateur de l'application utilise le mot de passe admin123 , facilement devinable. Une simple attaque par dictionnaire ou une tentative manuelle suffit à compromettre le compte le plus privilégié de l'application. L'adresse e-mail de l'administrateur admin@juice-sh.op est également prévisible.
Risque	Probabilité : Très élevée – Les attaques par dictionnaire et par force brute sont automatisées et triviales à mettre en œuvre. Impact : Critique – La compromission du compte administrateur donne un contrôle total sur l'application, les données utilisateurs et la configuration.
Composant affecté	Formulaire de connexion (/login) – compte admin@juice-sh.op
Outils utilisés	Tests manuels
Références	OWASP Top 10 A7:2021-Identification and Authentication Failures,

Preuve :

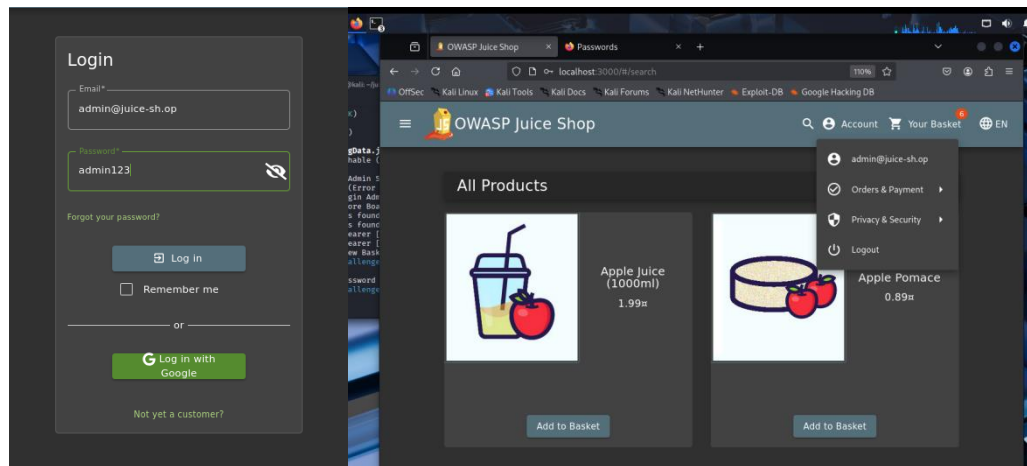


Figure 5 : Connexion réussie avec admin@juice-sh.op / admin123

Mesures correctives :

- Imposer une politique de mots de passe forts : minimum 12 caractères, mélange de majuscules, minuscules, chiffres et caractères spéciaux.
- Mettre en place un mécanisme de verrouillage de compte après un nombre limité de tentatives échouées.
- Activer l'authentification multi-facteurs (MFA) pour tous les comptes à privilèges élevés.
- Utiliser un algorithme de hachage robuste et salé (bcrypt, Argon2) pour le stockage des mots de passe.

**4. Constat JUICY-004: Exposition de données confidentielles -
Répertoire FTP(Élevée)**

Description	Le répertoire /ftp est accessible sans aucune exigence d'authentification. pu parcourir ce répertoire et visualiser tous les fichiers qu'il contient. Cette exposition pourrait entraîner la divulgation d'informations sensibles, y compris des fichiers de configuration, des fichiers de sauvegarde et d'autres données confidentielles.
Risque	Probabilité : Élevée – Les outils d'énumération de répertoires découvriraient rapidement ce répertoire exposé. Impact : Élevé – Des fichiers sensibles pourraient être consultés par des parties non autorisées, entraînant des violations de données.
Composant affecté	Répertoire /ftp
Outils utilisés	Navigateur Web http://localhost:3000/ftp

Preuve:

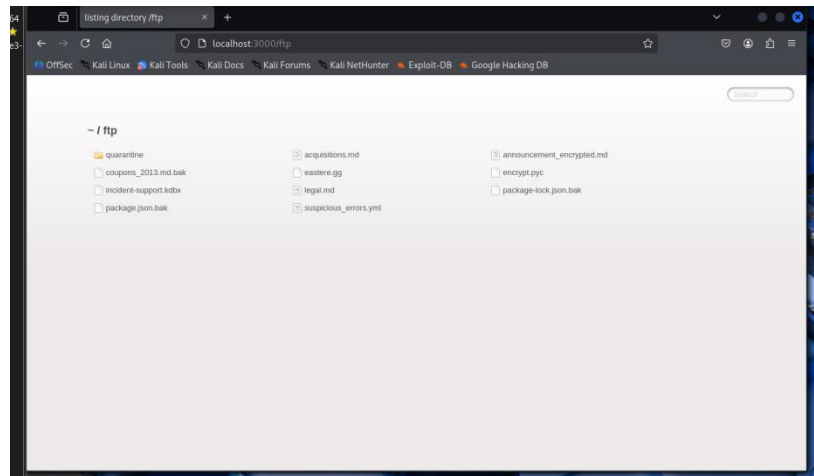


Figure 6: Accès non authentifié au répertoire /ftp* [Liste du répertoire montrant plusieurs fichiers]

Mesures correctives:

- Mettre en œuvre des contrôles d'accès appropriés pour tous les répertoires.
- Restreindre l'accès à /ftp aux seuls utilisateurs authentifiés et autorisés.
- Si le répertoire n'est pas nécessaire à des fins opérationnelles, le supprimer complètement.
- Effectuer des examens réguliers des contrôles d'accès

5. Constat JUICY-005 : Broken Access Control – Accès au panier d'un autre utilisateur (Élevée)

Description	L'API REST de Juice Shop ne vérifie pas que l'utilisateur authentifié est bien le propriétaire du panier demandé. En modifiant simplement l'identifiant dans la requête GET /rest/basket/{id}, il est possible d'accéder au panier de n'importe quel autre utilisateur sans aucune autorisation.
--------------------	--

Risque	Probabilité : Élevée – La manipulation d’identifiants dans les URLs est une technique d’attaque triviale et bien connue. Impact : Élevé – Un attaquant peut consulter le contenu du panier d’autres clients, révélant des données personnelles et des habitudes d’achat.
Composant affecté	API REST : /rest/basket/{id}
Outils utilisés	Tests manuels, navigateur web
Références	OWASP Top 10 A1:2021-Broken Access Control

Preuve :

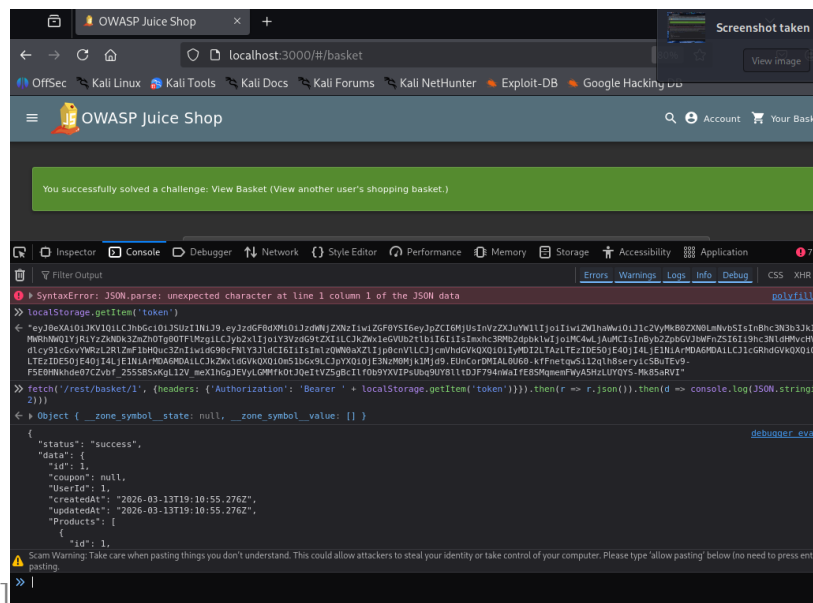


Figure 7 : Accès au panier de l'utilisateur ID 1 depuis le compte user2@test.com

Mesures correctives :

- Implanter un contrôle d'autorisation côté serveur vérifiant que l'utilisateur en session est le propriétaire de la ressource demandée.
- Ne jamais se fier aux identifiants fournis par le client pour déterminer les droits d'accès.

6. Constat JUICY-006 : Broken Access Control – Accès à la page d’administration (Élevée)

Description	La route /#/administration est accessible à tout utilisateur authentifié, quelle que soit son role. Aucun contrôle côté serveur ne vérifie les privilèges administrateur avant d’afficher le panneau de gestion.
Risque	Probabilité : Élevée – La route est découvrable en analysant les fichiers JavaScript Angular de l’application. Impact : Élevé – Un attaquant peut accéder à la liste complète des utilisateurs, gérer les commandes et les avis clients.
Composant affecté	Route Angular : /#/administration
Outils utilisés	Navigateur web, analyse du code source JavaScript
Références	OWASP Top 10 A1:2021-Broken Access Control,

Preuve :

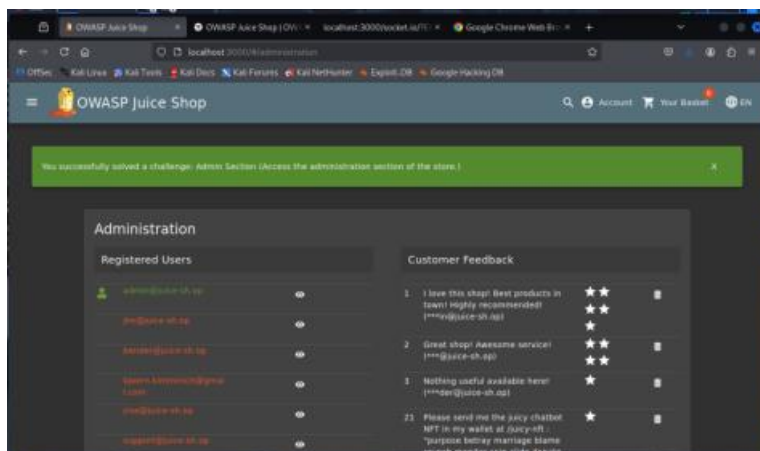


Figure 8 : Accès au panneau d’administration avec un compte standard – à compléter

Mesures correctives :

- Implémenter un contrôle d'accès basé sur les rôles (RBAC) côté serveur pour toutes les routes sensibles.
- Ne pas exposer les routes d'administration dans le bundle JavaScript public.
- Vérifier le rôle de l'utilisateur à chaque appel API, indépendamment du filtrage côté client.

7. Constat JUICY-007 : Mauvaise configuration de sécurité - Exposition du tableau des scores (Moyenne)

Description	Le tableau des scores, situé à l'adresse http://localhost:3000/#!/score-board , s'est avéré être accessible publiquement. Cette page répertorie tous les défis de l'application et fournit aux attaquants une feuille de route complète des vulnérabilités potentielles à exploiter
Risque	Probabilité : Moyenne – Les attaquants peuvent découvrir ceci via l'énumération de répertoires ou l'analyse JavaScript côté client. Impact : Moyen – L'exposition du tableau des scores révèle la surface d'attaque de l'application et les vulnérabilités prioritaires.
Composant affecté	Route /#!/score-board
Outils utilisés	Navigateur Web, analyse JavaScript

Références

OWASP Top 10 A6:2017-Mauvaise configuration de sécurité

Preuve:

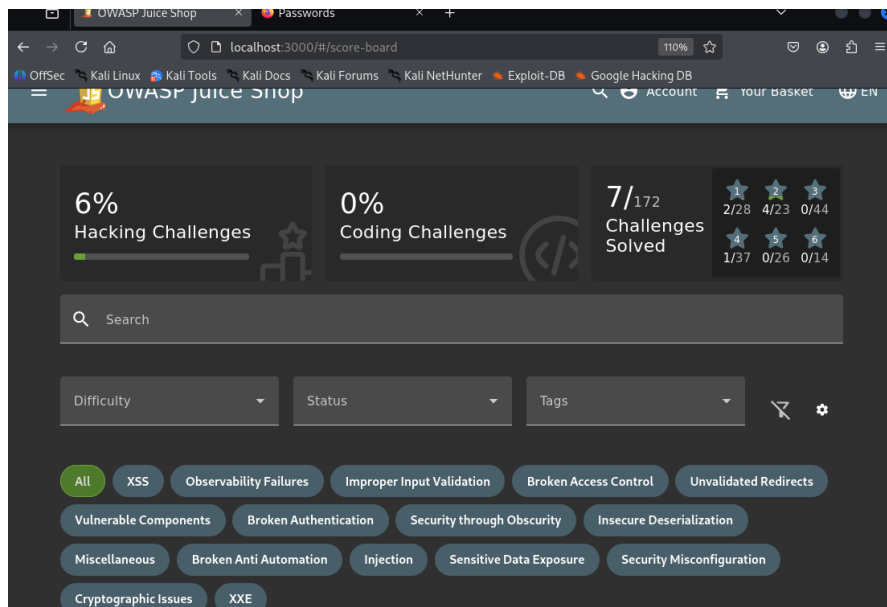


Figure 9 : Tableau des scores accessible publiquement

Mesures correctives:

- Supprimer ou protéger par mot de passe le tableau des scores dans les environnements de production.
- Mettre en œuvre des drapeaux de configuration spécifiques à l'environnement pour activer les fonctionnalités de débogage uniquement dans les déploiements hors production

8. Constat JUICY-008 : Exposition de la documentation de l'API (Moyenne)

Description	La documentation de l'API s'est avérée accessible à l'adresse /api-docs sans aucune exigence d'authentification. Cette documentation révèle tous les points d'accès API disponibles, les paramètres attendus, les structures de requête et les formats de réponse
Risque	Probabilité : Élevée – Les scanners automatisés et les tests manuels identifient rapidement la documentation API exposée. Impact : Moyen – L'exposition complète des points d'accès API permet des attaques ciblées.
Composant affecté	Point d'accès /api-docs
Outils utilisés	Navigateur Web
Références	OWASP Top 10 A3:2017-Exposition de données sensibles

Preuve :

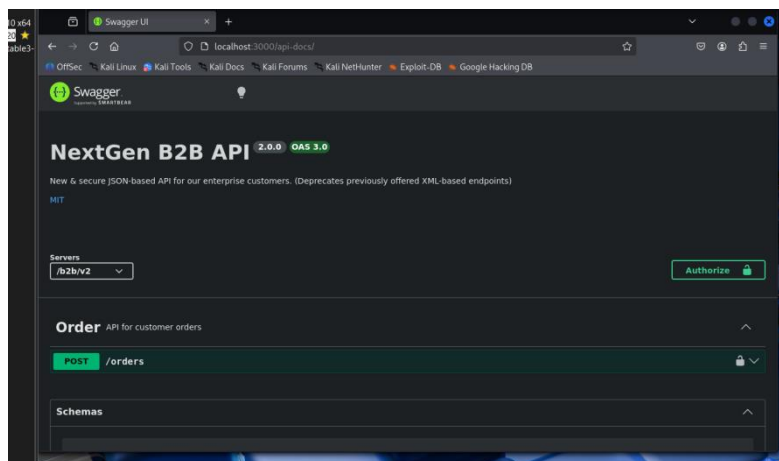


Figure 10 : Documentation API accessible publiquement

Mesures correctives ;

- Restreindre l'accès à la documentation de l'API aux seuls utilisateurs authentifiés et autorisés.
- Mettre en œuvre des contrôles d'accès basés sur les rôles pour la documentation.
- Envisager de déplacer la documentation vers un portail développeur interne avec une authentification appropriée.

9. Constat JUICY-009 : Cross-Site Scripting (XSS) – Champ de recherche (Moyenne)

Description	Le champ de recherche de l'application est vulnérable à une attaque Cross-Site Scripting (XSS) réfléchi. En injectant la charge utile <iframe src="javascript:alert(`1`)" /> dans le paramètre de recherche, la charge utile est retournée non filtrée dans la réponse, provoquant l'exécution du script dans le navigateur de la victime.
Risque	Probabilité : Élevée – Les charges utiles XSS réfléchies sont facilement détectables et exploitables via des liens piégés envoyés aux utilisateurs. Impact : Moyen – Un attaquant peut exécuter du code arbitraire dans le navigateur d'un utilisateur, voler des cookies de session, rediriger la victime ou dégrader l'interface de l'application.
Composant affecté	Champ de recherche (/#/search?q=)
Outils utilisés	Tests manuels, Navigateur Web

Références	OWASP Top 10 A7:2017-Cross-Site Scripting (XSS)
------------	---

Preuve :

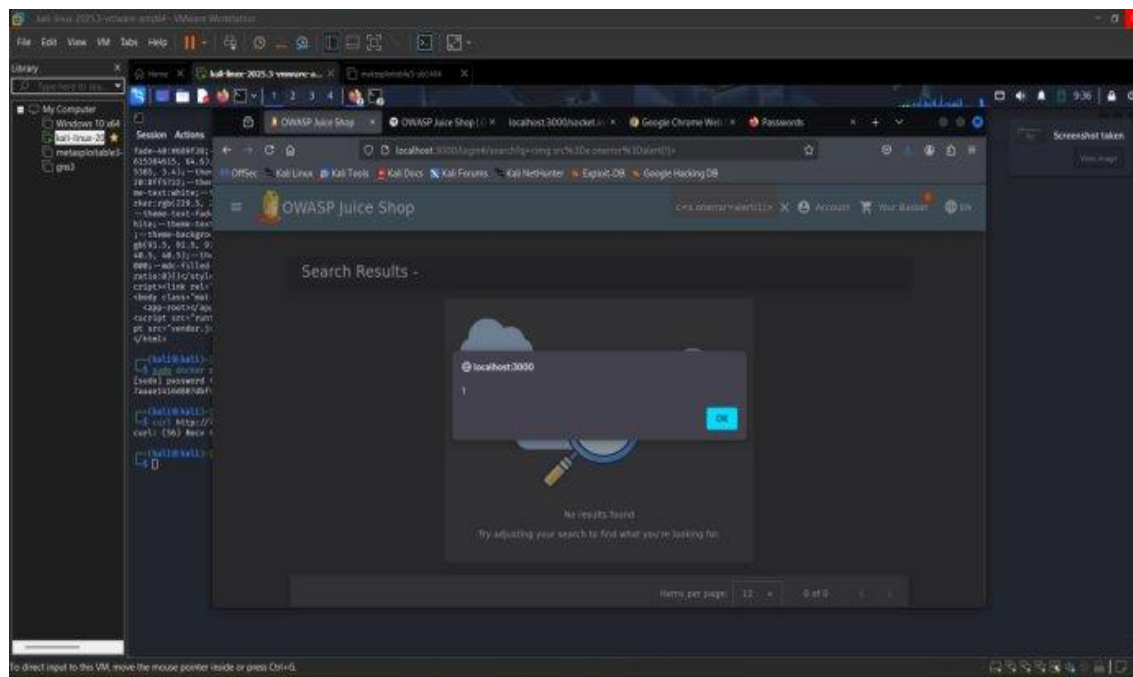


Figure 11: Exécution de la charge utile XSS via le champ de recherche

Mesures correctives :

- Mettre en œuvre un encodage contextuel des sorties (HTML encoding) pour toutes les données affichées dans le navigateur.
- Valider et assainir toutes les entrées utilisateur côté serveur ; rejeter les balises HTML et les schémas d'URI dangereux (javascript:).
- Déployer une politique de sécurité du contenu (Content Security Policy – CSP) stricte pour limiter les sources de scripts autorisées.
- Utiliser des bibliothèques de sécurité éprouvées (DOMPurify) pour assainir tout contenu HTML rendu dynamiquement.

10. Constat JUICY-010 : Cross-Site Request Forgery (CSRF) – Soumission de feedback (Moyenne)

Description	Le point d'accès POST /api/Feedbacks accepte un champ UserId contrôlable par l'utilisateur. En modifiant ce champ dans la requête, il est possible de soumettre un avis négatif ou un commentaire au nom de n'importe quel autre utilisateur de l'application, sans son consentement.
Risque	Probabilité : Moyenne – L'exploitation nécessite qu'un utilisateur authentifié soit amené à visiter une page malveillante. Impact : Moyen – Un attaquant peut nuire à la réputation d'autres utilisateurs en publiant des commentaires en leur nom.
Composant affecté	API REST : POST /api/Feedbacks
Outils utilisés	Tests manuels
Références	OWASP Top 10 A1:2021-Broken Access Control,

Preuve :

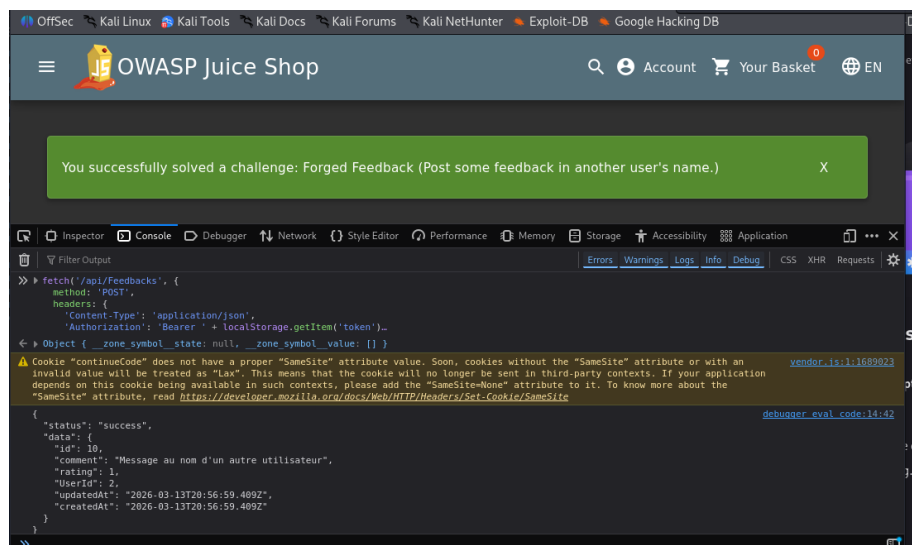


Figure 12 : Requête POST /api/Feedbacks avec UserId manipulé

Mesures correctives :

- Ignorer le champ UserId fourni par le client et déduire l'identité de l'utilisateur uniquement depuis le token de session côté serveur.
- Implémenter des jetons CSRF (anti-CSRF tokens) synchronisés pour tous les formulaires et requêtes état-modifiantes.
- Valider l'en-tête Origin et Referer pour les requêtes sensibles.

IV. Conclusion:

L'exposition de données sensibles et les mauvaises configurations élargissent considérablement la surface d'attaque. Ces résultats démontrent l'absence de contrôles de sécurité fondamentaux sur l'application.